

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 –DAST SIMULATION

Course Code:	CSA5803	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO3 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2
Weightage:	30% of CIA	Total Marks:	100
CO3 Statement:	Simulate Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA).		
Student Name:	S EZHILARASI	Reg. No.:	192521187
Section / Batch:	BTECH IT - B	Date:	13-08-2026

Instructions to Students

- Ensure all diagrams and workflow illustrations are **original, clear, and professionally formatted**.
- Include **all editable source files** (such as .yaml, .py, requirements.txt, .pre-commit-config.yaml, and any diagram source files if applicable).
- Submit **both** the **GitHub repository link** and the **final PDF report** through **Google Classroom** before the specified deadline.
- Verify that the GitHub repository is accessible and contains all required project files, screenshots, and documentation.

Question Type	Focus Area	Questions	Marks
DAST SIMULATION	Application based	Q1	Q1 –100
GRAND TOTAL:		100 Marks	

Code of Conduct

I, S EZHILARASI (192521187) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Ezhilarasi

CO3 AT2

SIMULATE DYNAMIC APPLICATION SECURITY TESTING (DAST) AND SOFTWARE COMPOSITION ANALYSIS (SCA).

Name: S EZHILARASI

Register Number: 192521187

Course Code: CSA5803

Course Name: DevSecOps Engineering and Security Applications

SIMULATE DYNAMIC APPLICATION SECURITY TESTING AND SOFTWARE COMPOSITION ANALYSIS

1. Title

Simulation of Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA) in a CI/CD Pipeline

2. Real-Life Scenario

Scenario: Online Banking Web Application

In a real-world software development environment, an online banking application is continuously developed and updated by developers. The application uses third-party libraries and open-source packages for authentication, database connectivity, user interfaces, and API communication.

A CI/CD pipeline automatically builds, tests, and deploys the application. For this simulation, a synthetic online banking web application is considered with intentionally simulated security vulnerabilities and outdated third-party dependencies.

DAST tests the running application from an external perspective, while SCA analyzes third-party and open-source dependencies. All users, transactions, source code, libraries, and findings are synthetic.

3. Problem Statement

Modern applications depend heavily on open-source libraries and third-party components. Vulnerabilities may also exist in the running application. If these issues are not identified before deployment, they may introduce security risks. Therefore, both DAST and SCA are included in the CI/CD pipeline.

- Build the application.
- Check third-party dependencies using SCA.
- Deploy the application to a test environment.
- Perform DAST on the running application.
- Identify and classify security vulnerabilities.
- Apply security gates.
- Block insecure releases.
- Generate security reports.
- Rescan after remediation.

4. Objectives

The main objective is to simulate DAST and SCA as automated security stages in a CI/CD pipeline.

Specific Objectives

1. Understand the purpose of DAST and SCA.
2. Integrate security testing into a CI/CD workflow.
3. Identify vulnerabilities in a running web application using DAST.
4. Identify vulnerable third-party dependencies using SCA.
5. Classify vulnerabilities based on severity.
6. Implement security gates.
7. Prevent vulnerable applications from being deployed.
8. Simulate remediation and rescanning.

9. Compare results before and after remediation.

5. DAST and SCA in DevSecOps

DAST

DAST examines the application while it is running and interacts with URLs, forms, APIs, authentication pages, and other exposed functionality.

DAST focuses on:

- Running application
- HTTP requests and responses
- Input validation
- Authentication
- Session handling
- Security headers
- Web vulnerabilities

SCA

SCA examines third-party and open-source components used by the application.

SCA focuses on:

- Package names and versions
- Known vulnerabilities
- Outdated dependencies
- License information
- Dependency risk

6. Real-Life CI/CD Pipeline

Developer → GitHub → Build → SCA Scan → Unit Test → Deploy → DAST Scan → Security Gate → Report → Production

Stage 1 – Code Commit

Developer pushes the latest code to GitHub.

Stage 2 – Build

Application is built and dependencies are installed.

Stage 3 – SCA Scan

Dependency files are analyzed for known vulnerabilities.

Stage 4 – Automated Testing

Unit and functional tests are executed.

Stage 5 – Test Deployment

Application is deployed to a controlled test environment.

Stage 6 – DAST Scan

Running application is dynamically scanned.

Stage 7 – Security Gate

SCA and DAST results are evaluated.

Stage 8 – Pipeline Decision

Critical or High findings above the allowed threshold cause failure.

Stage 9 – Report Generation

Combined security report is generated.

Stage 10 – Production Deployment

Application proceeds only when the security gate is passed.

7. Application Used for the Simulation

The simulated application is an Online Banking Web Application.

Module	Function
Login	User authentication
Dashboard	Account information
Account	Balance information
Transfer	Money transfer
Transactions	Transaction history
Profile	User information
Logout	Session termination

8. Synthetic SCA Dependency Dataset

ID	Dependency	Version	Simulated Issue	Severity
S001	Express	4.17.1	Known vulnerability	High
S002	Lodash	4.17.15	Prototype-related vulnerability	High
S003	Axios	0.21.0	Security vulnerability	Critical
S004	jQuery	3.4.1	Outdated version	Medium
S005	Moment	2.24.0	Outdated dependency	Low

These package versions are synthetic inputs for the simulation and are not intended to represent a live vulnerability database result.

9. SCA Simulation Steps

Step 1 – Identify Dependencies

The application dependency file is analyzed.

```
express@4.17.1
lodash@4.17.15
axios@0.21.0
jquery@3.4.1
moment@2.24.0
```

Step 2 – Build Dependency Inventory

Total Dependencies: 5

Direct Dependencies: 5

Step 3 – Compare Dependencies with Vulnerability Data

```
express@4.17.1 → High Risk
lodash@4.17.15 → High Risk
axios@0.21.0 → Critical Risk
jquery@3.4.1 → Medium Risk
moment@2.24.0 → Low Risk
```

Step 4 – Generate SCA Findings

Dependency	Version	Severity	Status
------------	---------	----------	--------

Express	4.17.1	High	Open
Lodash	4.17.15	High	Open
Axios	0.21.0	Critical	Open
jQuery	3.4.1	Medium	Open
Moment	2.24.0	Low	Open

10. SCA Result

SCA SCAN COMPLETED

Dependencies Scanned : 5

Critical : 1

High : 2

Medium : 1

Low : 1

Total Findings : 5

The simulator identifies Axios 0.21.0 as a Critical dependency finding. The security gate may fail depending on the configured policy.

11. DAST Simulation

After the build and dependency stages, the application is deployed into a controlled test environment.

Application Started

Test URL: http://localhost:3000

12. DAST Scan Steps

Step 1 – Start Application

Start the synthetic banking application on the test server.

Step 2 – Discover Application Endpoints

Identify available application pages.

Step 3 – Send Requests

Send HTTP requests to the application.

Step 4 – Analyze Responses

Analyze responses for possible security weaknesses.

Step 5 – Generate Findings

Generate the simulated vulnerability findings.

```
/login
/dashboard
/profile
/search
/transfer
/transactions
GET /login
GET /profile
GET /search
GET /transfer
GET /transactions
```

ID	URL	Vulnerability	Severity
D001	/login	Weak Authentication	High
D002	/profile	Cross-Site Scripting	High
D003	/search	SQL Injection Indicator	Critical

D004	/transfer	Missing Input Validation	Medium
D005	/login	Missing Security Headers	Medium
D006	/session	Weak Session Configuration	High
D007	/error	Information Disclosure	Low

13. DAST Result

DAST SCAN COMPLETED

URLs Scanned : 12
 Requests Sent : 150
 Critical : 1
 High : 3
 Medium : 2
 Low : 1
 Total Findings : 7

14. Combined DAST + SCA Result

Security Test	Critical	High	Medium	Low	Total
SCA	1	2	1	1	5
DAST	1	3	2	1	7
Total	2	5	3	2	12

The application has 12 simulated security findings.

15. Security Gate

```
IF Critical > 0
  Pipeline = FAILED
ELSE IF High > 0
  Pipeline = FAILED
ELSE
  Pipeline = PASSED
Critical = 2
High = 5
```

Therefore: Security Gate = FAILED

16. Pipeline Failure

```
BUILD      ✓ PASSED
SCA        ✓ COMPLETED
TEST       ✓ PASSED
DEPLOY     ✓ PASSED
DAST       ✓ COMPLETED
SECURITY   ✗ FAILED
PRODUCTION ✗ BLOCKED
```

This prevents the vulnerable application from reaching production.

17. Remediation

SCA Remediation

Outdated dependencies are upgraded to safer versions.

Before:
 axios@0.21.0

After:

axios@latest-secure-version

DAST Remediation

- Improve authentication controls.
- Validate user input.
- Encode output.
- Add security headers.
- Improve session configuration.
- Prevent sensitive information from appearing in error messages.

18. Rescan After Remediation

Developer Fix

↓

Git Commit

↓

Git Push

↓

CI/CD Pipeline

↓

SCA Rescan

↓

DAST Rescan

19. SCA Rescan Result

Severity	Before	After
Critical	1	0
High	2	0
Medium	1	1
Low	1	1
Total	5	2

20. DAST Rescan Result

Severity	Before	After
Critical	1	0
High	3	0
Medium	2	1
Low	1	1
Total	7	2

21. Final Security Result

SCA Critical : 0

SCA High : 0

DAST Critical: 0

DAST High : 0

Security Gate: PASSED

The application is now allowed to proceed toward production deployment.

22. Before and After Comparison

Parameter	Before Remediation	After Remediation
SCA Findings	5	2
DAST Findings	7	2
Critical Findings	2	0
High Findings	5	0
Medium Findings	3	2

Low Findings	2	2
Total Findings	12	4
Security Gate	Failed	Passed
Production Deployment	Blocked	Allowed

23. Proposed Simulation Dashboard

CI/CD Pipeline Status

BUILD ✓ PASSED
SCA ✓ PASSED
TEST ✓ PASSED
DEPLOY ✓ PASSED
DAST ✓ PASSED
SECURITY ✓ PASSED
PRODUCTION ✓ ALLOWED

SCA Dashboard

DEPENDENCIES SCANNED : 5
CRITICAL : 0
HIGH : 0
MEDIUM : 1
LOW : 1

DAST Dashboard

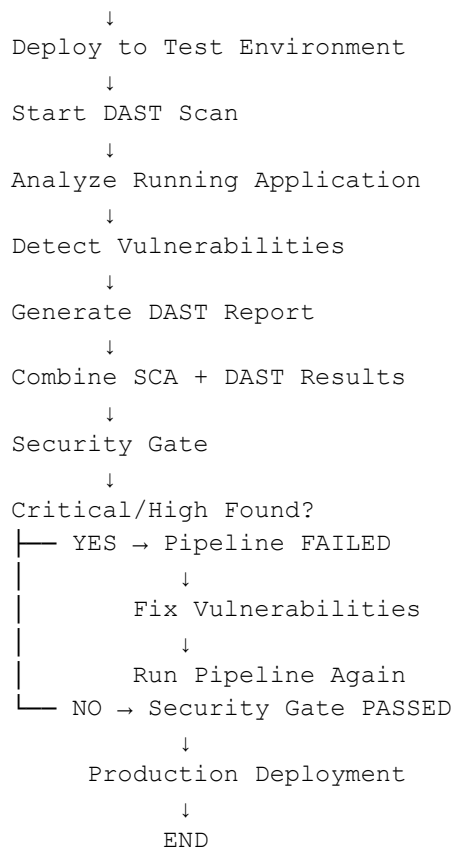
URLs SCANNED : 12
REQUESTS SENT: 150
CRITICAL : 0
HIGH : 0
MEDIUM : 1
LOW : 1

Simulation Controls

[Start Pipeline]
[Run SCA]
[Run DAST]
[View Findings]
[Fix Issues]
[Rescan]
[Reset]

24. Complete DAST + SCA CI/CD Workflow

START
↓
Developer Writes Code
↓
Push Code to GitHub
↓
CI/CD Pipeline Triggered
↓
Build Application
↓
SCA Scan
↓
Vulnerable Dependencies?
├─ YES → Security Alert
└─ NO → Run Unit Tests



25. Expected Outcome

The simulation is expected to demonstrate the combined use of DAST and SCA in a CI/CD pipeline.

10. Build the synthetic application.
11. Scan third-party dependencies using SCA.
12. Identify vulnerable dependencies.
13. Deploy the application to a test environment.
14. Perform DAST on the running application.
15. Identify application-level security vulnerabilities.
16. Classify vulnerabilities according to severity.
17. Generate SCA and DAST reports.
18. Combine the security findings.
19. Apply a CI/CD security gate.
20. Block an insecure application.
21. Simulate remediation.
22. Perform SCA and DAST rescans.
23. Verify that security issues have been reduced.
24. Allow the application to proceed after passing the security gate.

26. Conclusion

This project demonstrates the integration of Dynamic Application Security Testing (DAST) and Software Composition Analysis (SCA) into a CI/CD pipeline using a realistic online banking scenario.

SCA identifies vulnerabilities present in third-party and open-source dependencies, while DAST identifies security weaknesses in the running application.

By combining both techniques, the CI/CD pipeline provides a broader security assessment of the application. The security gate prevents applications containing unacceptable security risks from being deployed.

After remediation, the application is rescanned using both SCA and DAST. Once the critical and high-severity issues are resolved, the security gate is passed and the application can proceed toward production.

Final Result: The proposed simulation successfully demonstrates how DAST and SCA can work together in a CI/CD pipeline to identify vulnerabilities, prevent insecure releases, support remediation, and improve the overall security of a web application.